



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 09_2

NOMBRE COMPLETO: Judith Mora Ortega

N° de Cuenta: 318006538

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 07

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 03 Febrero 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

Objetivos de aprendizaje

I. Objetivos generales:

- a. El alumno aprenderá y comprenderá la creación de animaciones mediante el uso de banderas, condicionales, funciones y ecuaciones, la cuales nos permitirán simular el movimiento de objetos y crear escenas interactivas animadas en tiempo real

II. Objetivos específicos:

- a. El alumno comprenderá cómo crear una animación haciendo uso de transformaciones básicas (escala, rotación o traslación) y condicionales y lo aplicará a objetos tridimensionales, topología y/o textura de un objeto.
- b. El alumno comprenderá cómo crear animaciones haciendo uso de funciones y ecuaciones y lo aplicará a objetos tridimensionales , topología y/o textura de un objeto.

Desarrollo de práctica

1. Primero hice que el contador de los números fuera visible para el ojo humano

```
//Cambiar el tiempo de los numeros
float anguloVaria = 0.0f;
float toffsetNumerocambiarTimer = 0.0f; // temporizador para cambiar número
float toffsetNumerocambiarInterval = 8.0f; // intervalo en segundos (ajustar para más/menos velocidad)
```

2. Con el modelo cargado el profesor del Aeolipile al prender el fuego y se detiene cuando el fuego se extingue

```
//Aeolipile
//Variables para el fuego animado y el giro del Aeolipile.
// El fuego se activa con la tecla F, vive un tiempo definido y luego se extingue.
// El Aeolipile gira mientras haya fuego activo.
bool fuegoActivo = false; // bandera: true mientras el fuego está encendido
float fuegoTimer = 10.0f; // acumula el tiempo transcurrido desde que se encendió
float fuegoDuracion = 550.0f; // segundos que dura el fuego antes de extinguirse
float fuegoAlpha = 0.0f; // opacidad actual del fuego (0=apagado, 1=pleno)
float rotAeolipile = 0.0f; // ángulo de giro acumulado de la esfera del Aeolipile
float velAeolipile = 5.0f; // grados por segundo cuando el fuego está activo
//Fin
```

Declaramos las variable globales

Nos agregue un peque modelo de un cono para hacer de base al fuego, ya que puede cargar mi animación de blender

```
// Tecla F enciende el fuego y reinicia su duración
if (mainWindow.getKeys()[GLFW_KEY_F])
{
    fuegoActivo = true;
    fuegoTimer = 0.0f;
}

//Avanzar el fuego cada frame
if (fuegoActivo)
{
    fuegoTimer += deltaTime;

    // Encendido suave: sube de 0 a 1 en los primeros 0.3 segundos
    if (fuegoTimer < 0.3f)
        fuegoAlpha = fuegoTimer / 0.3f;
    // Llama estable
    else if (fuegoTimer < fuegoDuracion - 2.0f)
        fuegoAlpha = 1.0f;
    // Extinción suave: baja de 1 a 0 en los últimos 2 segundos
    else if (fuegoTimer < fuegoDuracion)
        fuegoAlpha = 1.0f - ((fuegoTimer - (fuegoDuracion - 2.0f)) / 2.0f);
    // Fuego apagado completamente
    else
    {
        fuegoActivo = false;
        fuegoAlpha = 0.0f;
    }
}
```

Aquí agregue que con F se prenda el fuego y parpadee simulando el fuego.

3. Agregamos el modelo de humo que hice paso blender aquí se le dieron todos parámetros de animación

```

// GiraAeolipile
float velGiroActual = fuegoActivo ? velAeolipile * fuegoAlpha : 5.0f;
rotAeolipile += velGiroActual * deltaTime;
if (rotAeolipile >= 360.0f)
    rotAeolipile -= 360.0f;

// El Aeolipile gira solo mientras hay fuego
// fuegoAlpha como multiplicador hace que arranque y frene suavemente
if (fuegoActivo)
{
    // Instancia 1: sube continuamente y se reinicia
    humoOffset1 += humoVelocidad * fuegoAlpha * deltaTime;
    if (humoOffset1 >= humoReset)
        humoOffset1 = 0.0f;

    // Instancia 2: desfasada a la mitad del ciclo para que no suban juntas
    humoOffset2 += humoVelocidad * fuegoAlpha * deltaTime;
    if (humoOffset2 >= humoReset)
        humoOffset2 = 0.0f;

    // Opacidad: aparece al salir y se desvanece al llegar arriba
    // Instancia 1
    if (humoOffset1 < humoReset * 0.3f)
        humoAlpha1 = humoOffset1 / (humoReset * 0.3f); // fade in
    else if (humoOffset1 < humoReset * 0.7f)
        humoAlpha1 = 1.0f; // pleno
    else
        humoAlpha1 = 1.0f - ((humoOffset1 - humoReset * 0.7f) / (humoReset * 0.3f)); // fade out
}

```

```

// Multiplica por fuegoAlpha para que el humo también aparezca/desaparezca
// suavemente junto con el fuego
humoAlpha1 *= fuegoAlpha;
humoAlpha2 *= fuegoAlpha;
}
else
{
    // Fuego apagado: reiniciar humo
    humoOffset1 = 0.0f;
    humoOffset2 = humoReset * 0.5f; // desfase inicial para el próximo encendido
    humoAlpha1 = 0.0f;
    humoAlpha2 = 0.0f;
}
}

```

4. Para este punto hice ocupar dos modelos en Blender uno catapulta que la dividí y un balón.

```

// CATAPULTA: solo activa cuando el fuego está encendido
if (fuegoActivo)
{
    if (faseCatapulta == 0)
    {
        timerEspera += deltaTime;
        rotBrazo = 1.0f;
        if (timerEspera >= esperaInterval)
        {
            timerEspera = 0.0f;
            faseCatapulta = 1; // + AQUÍ estaba el bug, decía 0
        }
    }
    else if (faseCatapulta == 1)
    {
        rotBrazo += velBrazo * deltaTime;
        if (rotBrazo >= 70.0f)
        {
            rotBrazo = 70.0f;
            balonEnVuelo = true;
            rebotes = 0;
            balonX = 15.0f; // + igual que el translate X de la catapulta
            balonY = 2.5f;
            balonZ = 0.0f;
            balonVELY = balonVELYInicial;
            faseCatapulta = 2;
        }
    }
}

```

```

    else if (faseCatapulta == 2) // brazo baja lento
    {
        rotBrazo -= (velBrazo * 0.1f) * deltaTime;
        if (rotBrazo <= 1.0f)
        {
            rotBrazo = 1.0f;
            faseCatapulta = 3;
        }
    }
    else if (faseCatapulta == 3) // recarga y repite
    {
        timerEspera += deltaTime;
        if (timerEspera >= esperaInterval)
        {
            timerEspera = 0.0f;
            faseCatapulta = 0;
        }
    }
}
else
{
    faseCatapulta = 0;
    rotBrazo = 1.0f;
    timerEspera = 0.0f;
}

```

Esta parte nos el lanzamiento de balón además limitar de catapulta.

```

// BALÓN física parabólica con rebotes
if (balonEnVuelo)
{
    // Aplicar gravedad
    balonVelY += gravedad * deltaTime;

    // Mover balón
    balonZ += balonVelX * deltaTime;
    balonY += balonVelY * deltaTime;

    // Rebotar en el piso
    if (balonY <= pisoBayon)
    {
        balonY = pisoBayon;
        balonVelY = -balonVelY * energiaRebote; // rebota con menos energía
        rebotes++;

        // Cada rebote también pierde un poco de velocidad horizontal
        balonVelX *= 0.85f;

        if (rebotes >= maxRebotes)
        {
            // Terminó de rebotar: balón se detiene y desaparece
            balonEnVuelo = false;
            balonVelX = 4.0f; // restaurar para el próximo disparo
        }
    }
}
}

```

La parte de los rebotes y la trayectoria parabólica

Está aquí todas las modificaciones fueron el LOOP

Las siguientes modificaciones fueron Render

```

//AEOLIPILE
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -0.5f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Aeolipile_base_M.RenderModel();

model = glm::translate(model, glm::vec3(0.0f, 5.0f, 0.0f));
model = glm::rotate(model, rotAeolipile * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Aeolipile_M.RenderModel();

```

Se ajustan las ubicaciones de base como la bolita que gira

```

// HUMO: parte desde modelaux, hereda el giro del Aeolipile
if (fuegoActivo || humoAlpha1 > 0.0f || humoAlpha2 > 0.0f)
{
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);

    // Instancia 1 - tubo 1
    if (humoAlpha1 > 0.0f)
    {
        float escalaHumo1 = 0.001f + humoOffset1 * 0.005f;

        color = glm::vec3(0.6f, 0.6f, 0.6f) * humoAlpha1;
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));

        model = modelaux;
        model = glm::translate(model, glm::vec3(-1.0f, -0.8f, 0.0f));
        model = glm::rotate(model, -90.0f * toRadians, glm::vec3(0.0f, humoOffset1, 0.0f));
        model = glm::scale(model, glm::vec3(escalaHumo1, escalaHumo1 * 0.8f, escalaHumo1));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        plainTexture.UseTexture();
        Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
        Humo_M.RenderModel();
    }
}

```

```

// Instancia 2 - tubo 2 (lado opuesto)
if (humoAlpha2 > 0.0f)
{
    float escalaHumo2 = 0.001f + humoOffset2 * 0.005f;

    color = glm::vec3(0.5f, 0.5f, 0.5f) * humoAlpha2;
    glUniform3fv(uniformColor, 1, glm::value_ptr(color));

    model = modelaux;
    model = glm::translate(model, glm::vec3(0.5f, 0.5f, -0.0f));
    model = glm::rotate(model, +90.0f * toRadians, glm::vec3(0.0f, humoOffset2, 0.0f));
    model = glm::scale(model, glm::vec3(escalaHumo2, escalaHumo2 * 0.8f, escalaHumo2));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    plainTexture.UseTexture();
    Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
    Humo_M.RenderModel();
}

glDisable(GL_BLEND);
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
}

```

Darle color al humo, las animaciones correspondientes, y colocar las coordenadas de cada humo.

```

// CATAPULTA
// Posición base de toda la catapulta en la escena
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
modelaux = model; // guardamos la base para que brazo y balón hereden posición

// BASE
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(15.0f, -1.3f, 0.0f));
model = glm::scale(model, glm::vec3(0.1f, 0.1f, 0.1f));
modelaux = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.55f, 0.27f, 0.07f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
plainTexture.UseTexture();
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
Catapulta_Base.RenderModel();

// BRAZO
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 1.0f, 0.0f)); // pivote en espacio del modelo
model = glm::rotate(model, rotBrazo * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glm::mat4 modelBrazo = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.55f, 0.27f, 0.07f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
plainTexture.UseTexture();
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
Catapulta_Brazo.RenderModel();

```

Esta parte de la ubicación de los modelos cargados, colores, etc.

```

// BALÓN en cuchara
if (!balonEnVuelo)
{
    model = modelBrazo;
    model = glm::translate(model, glm::vec3(-18.0f, 65.0f, -52.0f));
    // escala absoluta para el balón
    model = glm::scale(model, glm::vec3(0.2f, 0.2f, 0.2f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    color = glm::vec3(1.0f, 1.0f, 2.0f);
    glUniform3fv(uniformColor, 1, glm::value_ptr(color));
    plainTexture.UseTexture();
    Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
    Balon_M.RenderModel();
}
else
{
    model = glm::mat4(1.0);
    model = glm::translate(model, glm::vec3(balonX, balonY, balonZ));
    model = glm::rotate(model, angulovariable * 200.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
    model = glm::scale(model, glm::vec3(0.05f, 0.05f, 0.05f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    color = glm::vec3(1.0f, 1.0f, 1.0f);
    glUniform3fv(uniformColor, 1, glm::value_ptr(color));
    plainTexture.UseTexture();
    Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
    Balon_M.RenderModel();
}

```

La parte que une la catapulta con la pelota para el lanzamiento


```

// Dibujar el fuego con Fuego_M, solo si está activo
if (fuegoAlpha > 0.0f)
{
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);

    // Parpadeo usando angulovaria
    float parpadeo = 0.55f + 0.15f * sinf(angulovaria * 8.0f);
    color = glm::vec3(1.0f, 0.4f * parpadeo, 0.0f); // naranja-rojo
    glUniform3fv(uniformColor, 1, glm::value_ptr(color));

    // La escala crece con fuegoAlpha: la llama "crece" al encenderse
    model = glm::mat4(1.0);
    model = glm::translate(model, glm::vec3(0.0f, 0.5f, 1.5f));
    model = glm::rotate(model, angulovaria * 1.0f * toRadians, glm::vec3(0.0f, 10.0f, 0.0f)); // rotación leve de la llama
    model = glm::scale(model, glm::vec3(fuegoAlpha * 0.8f, fuegoAlpha * 1.0f, fuegoAlpha * 0.8f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    plainTexture.UseTexture();
    Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
    Fuego_M.RenderModel();

    glDisable(GL_BLEND);

    // Restablecer color blanco
    color = glm::vec3(1.0f, 1.0f, 1.0f);
    glUniform3fv(uniformColor, 1, glm::value_ptr(color));
}

```

Y por último sigamos valores a posición el fuego, colores y la animación de fuego.

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

→ Los problemas que tuve fueron usar blender al cambiar los modelos a triángulos.

→ Manejar los diferentes materiales, hice un fuego desde 0 en blender con animación pero no puede ponerlo en Visual.

→ Las animaciones desde el código fueron complejas me costó mucho ubicar los modelos de blender en especial el humo.

→ Siento que en tiro parabólico me fallo no logre que se viera claramente cómo rebota



3.- Conclusión:

Me costó mucho la parte física, la parte de blender, además encontrar la coordenadas correctas para los modelos y escalarlos de manera correcta, la parte de humo y los movimientos que hace cada cosa me se dificulto a nivel código, también como importar de manera de correcta en blender, ya que luego no salían mis archivos completos,

1. Bibliografía en formato APA
Roque, J. (2026). *opengl3.3ydosmain* [Material de código]. UMAN.
2. **Cartoon bubble cloud.** (s/f). Cgtrader.com. Recuperado el 3 de mayo de 2026, de <https://www.cgtrader.com/items/3668640/download-page>
3. **Vintage leather ball.** (s/f). CGTrader. Recuperado el 3 de mayo de 2026, de <https://www.cgtrader.com/free-3d-models/sports/equipment/vintage-leather-ball>
4. **Catapult FREE low-poly.** (s/f). CGTrader. Recuperado el 3 de mayo de 2026, de

[https://www.cgtrader.com/free-3d-models/military/military-vehicle/cata
pult-622e2b53-51fb-450c-91ad-5acfaa576afc](https://www.cgtrader.com/free-3d-models/military/military-vehicle/cata-pult-622e2b53-51fb-450c-91ad-5acfaa576afc)